



МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ

Институт №3 — «Системы управления, информатика и электроэнергетика»
Кафедра 307 — «Цифровые технологии и информационные системы»

ОТЧЁТ
О ПРАКТИЧЕСКОЙ РАБОТЕ

по теме:

ПРИМЕНЕНИЕ ИИ ПРИ РАЗРАБОТКЕ ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ
МОДЕЛИРОВАНИЯ СИСТЕМ МАССОВОГО ОБСЛУЖИВАНИЯ

Принял:

Ст. преподаватель
кафедры 307

подпись, дата

Коробков М.А.

Выполнил:

Студент МЗО-321Б-23

подпись, дата

Елисеев П.А.

СОДЕРЖАНИЕ

Введение	3
1 Методика работы с ИИ-ассистентом	4
2 Техническое задание и архитектура	5
3 Реализация приложения	6
4 Итеративная доработка по обратной связи	7
5 Верификация результатов	7
6 Оценка эффективности применения ИИ	8
Заключение	9

ВВЕДЕНИЕ

Цель работы — на практике оценить, насколько современный агентный ИИ-ассистент Claude Code применим в полном цикле создания программного продукта от постановки задачи до работающего приложения. В качестве предметной области выбран анализ и построение технологических маршрутов, рассматриваемых как системы массового обслуживания (СМО). Разрабатываемое приложение QueuingFlow позволяет визуально собирать карту процесса (источники заявок, операции обслуживания, стоки), мгновенно получать аналитическую оценку загруженности узлов по теории сетей Джексона и проверять её пошаговой симуляцией. Работа состоит в том, что весь код приложения написан ИИ-ассистентом под управлением человека: роль автора заключалась в постановке задач, проверке результата в реальном приложении и формулировании правок.

1 Методика работы с ИИ-ассистентом

Взаимодействие строилось в формате диалога: человек формулировал намерение на естественном языке, ассистент предлагал решение (текст ТЗ, архитектуру, код), после чего человек запускал приложение, выявлял недостатки и формулировал следующую итерацию. Ассистент самостоятельно выполнял рутинные операции: создание файлов, запуск тестов и сборки, исправление ошибок. Общая схема процесса приведена на рисунке 1.

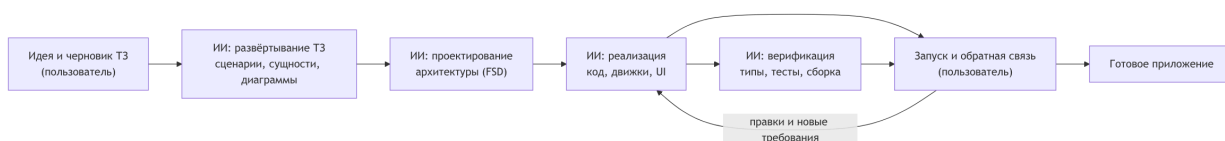


Рисунок 1 — Цикл разработки с ИИ-ассистентом

Разделение ответственности между человеком и ИИ показано в таблице 1.

Таблица 1 — Распределение задач между человеком и ассистентом

Этап	Человек	ИИ-ассистент
ТЗ	идея, требования, приоритеты	развёртывание ТЗ, сценарии, диаграммы
Архитектура	выбор стиля (FSD), ревизия	декомпозиция по слоям, границы
Реализация	приёмка, постановка правок	написание кода, движков, UI
Отладка	воспроизведение ошибок	локализация и исправление
Верификация	сценарии проверки	типизация, тесты, сборка

2 Техническое задание и архитектура

Исходными данными послужил краткий черновик идеи. Ассистент развернул его в полноценное техническое задание: описал четыре модуля системы (холст, JSON-редактор, аналитический и симуляционный движки), функциональные требования, сущности и структуру JSON-конфигурации, а также пользовательские сценарии и нештатные ситуации (нестабильность системы, некорректный ввод, недостижимые узлы). Вклад ИИ состоял в систематизации: из общего описания были получены измеримые требования и критерии приёмки.

Для реализации был выбран подход Feature-Sliced Design (FSD). Доменная модель вынесена в единственную сущность `process-model`, а аналитика и симуляция оформлены как прикладные сценарии (features) над ней — это сохраняет изоляцию слоёв. Структура слоёв показана на рисунке 2.

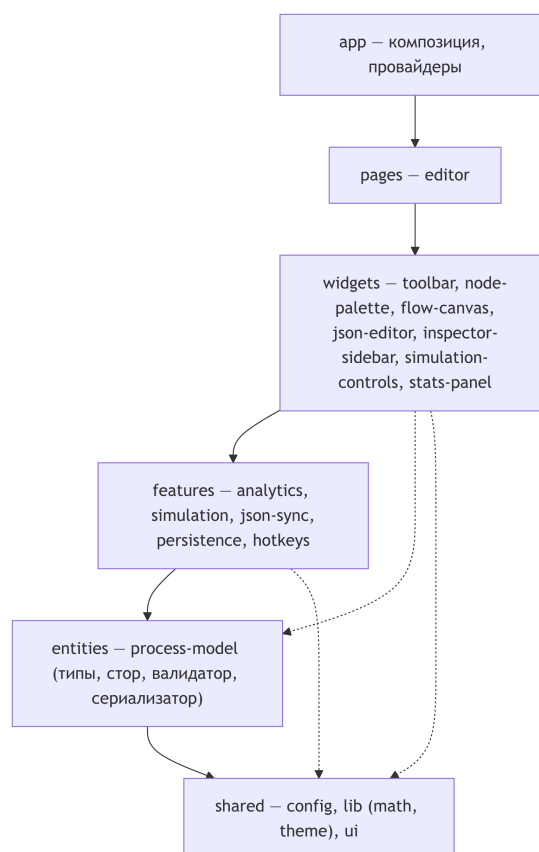


Рисунок 2 — Слои приложения (Feature-Sliced Design)

3 Реализация приложения

Технологический стек: React, TypeScript, React Flow (холст и графы), Monaco Editor (редактор JSON), Zustand (состояние), Vite и Bun (сборка и тесты). Общий вид интерфейса приведён на рисунке 3.

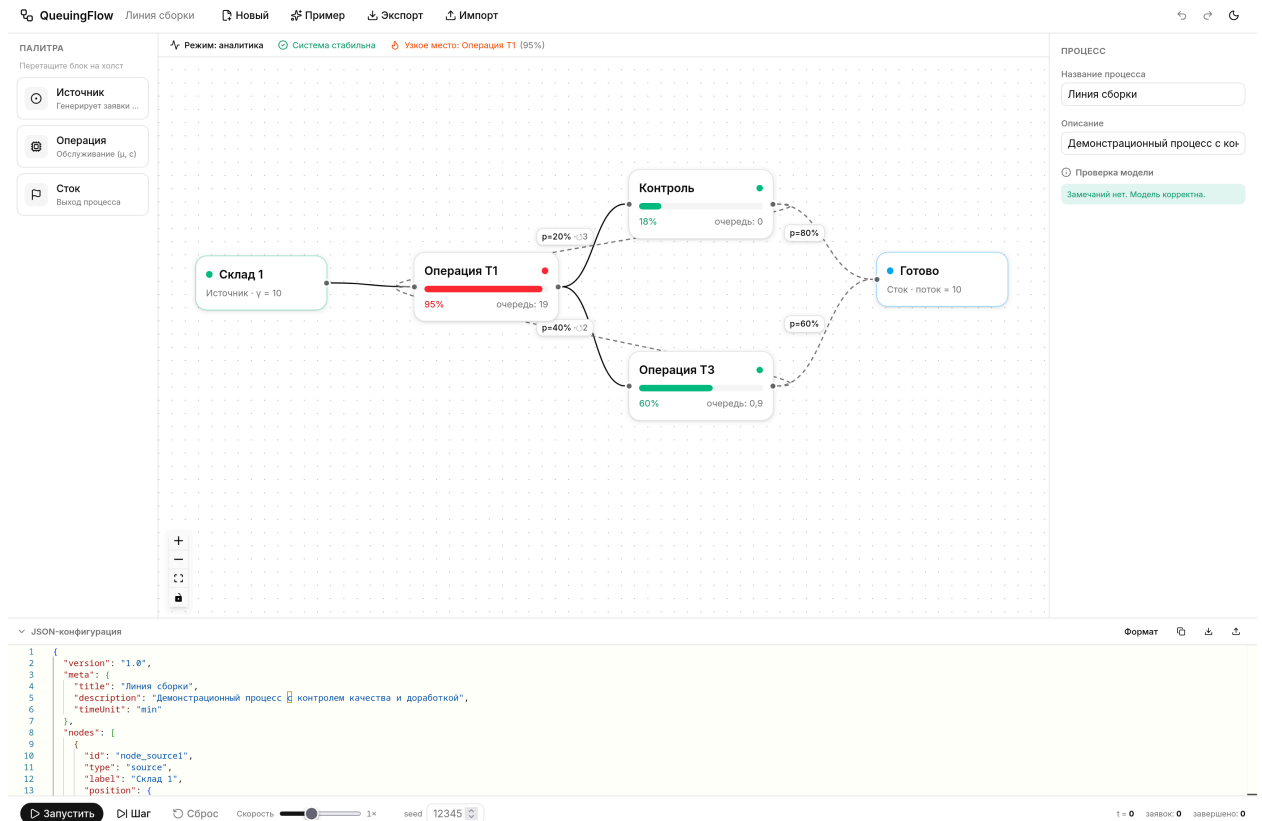


Рисунок 3 — Общий вид интерфейса QueuingFlow

Интерактивный холст. Реализован холст с перетаскиванием узлов из палитры, созданием связей, ветвлениями и циклами. Параметры узла раскрываются по трём уровням: на блоке (загрузка и очередь), во всплывающей подсказке (время ожидания, пропускная способность) и в боковой панели (полное редактирование).

Синхронизация с JSON. Граф является единственным источником истины, а холст и JSON-редактор его представления. Любое изменение схемы мгновенно отражается в JSON и наоборот; некорректный JSON не разрушает модель, а подсвечивается с описанием ошибки.

Аналитический движок. Реализован фоновый расчёт по модели открытой сети Джексона: решение уравнений трафика и формулы $M/M/c$ (загрузка ρ , длина очереди L_q , время ожидания W_q , время в системе W). Узлы

окрашиваются по степени загрузки, отдельно индицируется нестабильность ($\rho \geq 1$) и «узкое место».

Имитационная симуляция. Выполнена как дискретно-событийная модель: события (генерация, обслуживание, маршрутизация) обрабатываются по времени, заявки движутся по карте с учётом вероятностей ветвления, лимитов циклов и потерь. Управление — запуск, пауза, пошаговый режим, сброс и регулировка скорости.

4 Итеративная доработка по обратной связи

После первичной реализации приложение неоднократно запускалось и улучшалось по замечаниям. В числе доработок: исправление поведения кнопки пошагового режима; замена абстрактных «частиц» на полноценные заявки-сущности, привязанные к связям; корректный учёт потерь заявок (тупики и регулируемая вероятность потери); статистика по стокам; поддержка тёмной темы; горячая клавиша отмены (Ctrl+Z) и пауза по пробелу. Отдельно реализованы опорные точки маршрута связей: двойной клик добавляет перетаскиваемую точку, через которую связь изгибается, а заявки при симуляции движутся точно вдоль получившегося пути.

5 Верификация результатов

Качество контролировалось автоматически: строгая типизация TypeScript, юнит-тесты доменных движков и production-сборка. Аналитический движок проверялся на эталонных формулах M/M/1 и M/M/c, симулятор — на сходимости загрузки к аналитическому значению ρ , корректности лимита циклов и балансе заявок (создано = завершено + потеряно + в системе).

6 Оценка эффективности применения ИИ

Сильные стороны подхода: высокая скорость от идеи до работающего прототипа; систематизация требований и проектных решений; готовность ассистента самостоятельно запускать проверки и устранять ошибки; единообразие кода и соблюдение выбранной архитектуры.

Границы применимости: ассистенту требуется чёткая обратная связь от запуска реального приложения — визуальные и поведенческие дефекты человек выявлял сам; ряд решений (модель потери заявки, поведение анимации) уточнялся итеративно; финальную приёмку и постановку приоритетов выполнял человек. В целом связка «человек как заказчик и контролёр + ИИ как исполнитель» показала себя продуктивной для разработки приложения подобного масштаба.

ЗАКЛЮЧЕНИЕ

В ходе работы с применением ИИ-ассистента создано работающее веб-приложение *QueuingFlow*, реализующее визуальное построение СМО, фоновый аналитический расчёт по сети Джексона и имитационную симуляцию с анимацией заявок. Пройден полный цикл: техническое задание, архитектура, реализация, итеративная доработка и верификация. Подтверждено, что агентный ИИ способен выступать основным исполнителем при сохранении за человеком функций постановки задач и контроля качества.